

EECS 482

Lab 4: Semaphores; Thread Implementation



Administrivia

Today: 06/06

Project 2

Due 06/20

2 weeks!

Honor Code Reminder!

— — —

- Syllabus: *We encourage students to use online resources such as cppreference, Stack Overflow, and ChatGPT. However, you may only use these types of resources for general programming help (such as how to use a particular C++ feature); you may not use them to get help that is specific to the EECS 482 projects. **You may not copy-paste any part of the project handouts into a GenAI tool.** Be wary of using AI pair programmers (e.g., Copilot), as it may provide code that causes you to violate the honor code.*
- Do **not** use GenAI for help on EECS 482 project-specific questions
 - **OK:** C++ questions, debugging tips, practice exam questions, etc.
 - **NOT OK:** GitHub Copilot, directly generating project code, asking for **design ideas**, stuff you would put in a private piazza post, generating **test cases**, generating **verifiers/checkers**, generating **documentation**.
- Do **not** discuss implementations publicly to other groups.

P2 Progress (Suggestions)

— — —

1. `cpu::cpu()`
2. `thread::thread()`
3. `thread::yield()`
4. `mutex::mutex()`, `mutex::lock()`, `mutex::unlock()`, `mutex::~~mutex()`
5. `cv::cv()`, `cv::wait()`, `cv::signal()`, `cv::broadcast`, `cv::~~cv()`
6. `thread::join()`
7. `interrupt_enable/interrupt_disable`, timer interrupt handler
8. multiple CPUs (6-credit groups)

 You should be here

Agenda

1. Semaphore Exercises
2. Implementing threads
3. Project 2 tips
4. Testing in project 2

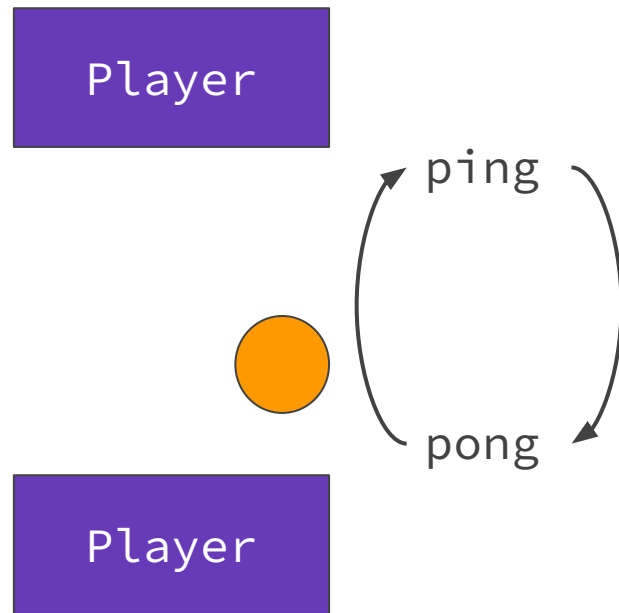
— — —

Ping Pong! Now with semaphores!

Reminder: Ping Pong Specification

- One thread for each player
- Players must take turns hitting the ball
- Each player hits ball 5 times
- Command line argument to control determinism
- *Bonus: Either player can start*

Use Semaphores!



Solution, if ping always starts

```
void ping(uintptr_t){
    uint hit_count = 0;
    while(hit_count < 5){
        ping_s.down();
        std::cout << "ping\n";
        hit_count++;
        pong_s.up();
    }
}
```

```
semaphore ping_s = 1;
semaphore pong_s = 0;
```

```
void pong(uintptr_t){
    uint hit_count = 0;
    while(hit_count < 5){
        pong_s.down();
        std::cout << "pong\n";
        hit_count++;
        ping_s.up();
    }
}
```

Allowing Either Thread to Go First

```
semaphore start_s = 1;  
bool first = true
```

```
start_s.down();  
if(first){  
    // if ping went first  
    first = false;  
}  
else{  
}  
start_s.up()
```

```
start_s.down();  
if(first){  
    // if pong went first  
    first = false;  
}  
else{  
}  
start_s.up()
```

Now, each thread can know which one was first, and previous logic can be mirrored.

Condensed Solution

— — —

```
semaphore first_s = 0;
semaphore second_s = 0;

semaphore start_s = 1;
bool first = true
```

```
void boot(uintptr_t) {
    thread ponger(player, 0);
    thread pinger(player, 1);
}
```

```
void player(uintptr_t arg) {
    uint hit_count = 0;
    bool im_first = false;

    start_s.down();
    if(first){
        im_first = true;
        first = false;
    }
    start_s.up();

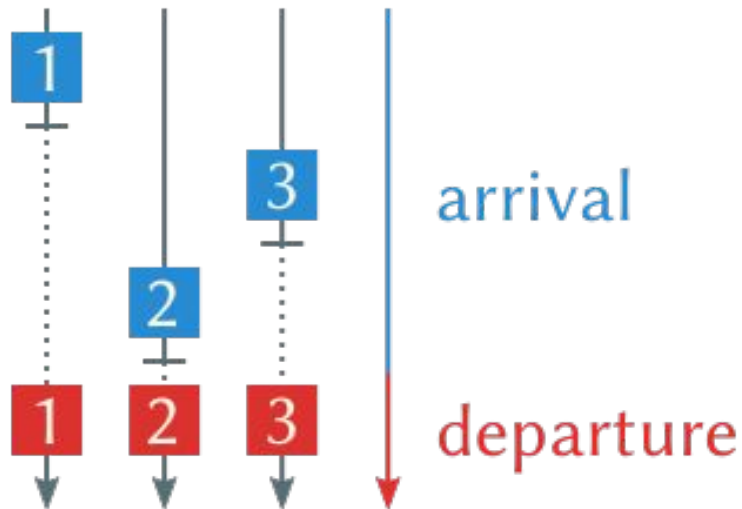
    while(hit_count < 5){
        if(im_first){
            printf(arg ? "ping\n" : "pong\n");
            first_s.up();
            second_s.down();
        }
        else{
            first_s.down();
            printf(arg ? "ping\n" : "pong\n");
            second_s.up();
        }
        hit_count++;
    }
}
```

Barriers

Barriers

Barriers allow a set of threads to synchronize their progress.

Threads "check in" or wait at a barrier and are only allowed to proceed past the barrier after all the threads have checked in.



Barriers

— — —



```
semaphore ready = 0;           // # of runners at starting line
semaphore started = 0;         // race has started
```

```
void runner() {
    ready.up();                 // arrive
    started.down();             // wait for race to start
}
```

```
void starter() {
    for (int i = 0; i < N; i++)
        ready.down();           // wait for N runners
    for (int i = 0; i < N; i++)
        started.up();           // start N runners
}
```

Threads

Two perspectives on threads

— — —

User Perspective:

Threads run in **parallel**

Thread states:

runnable, **blocked**

Kernel Perspective:

CPUs are finite

Need to **interleave** threads

Thread states:

running, **blocked**, **ready**

The goal of project 2 is to implement the thread library you used in project 1.

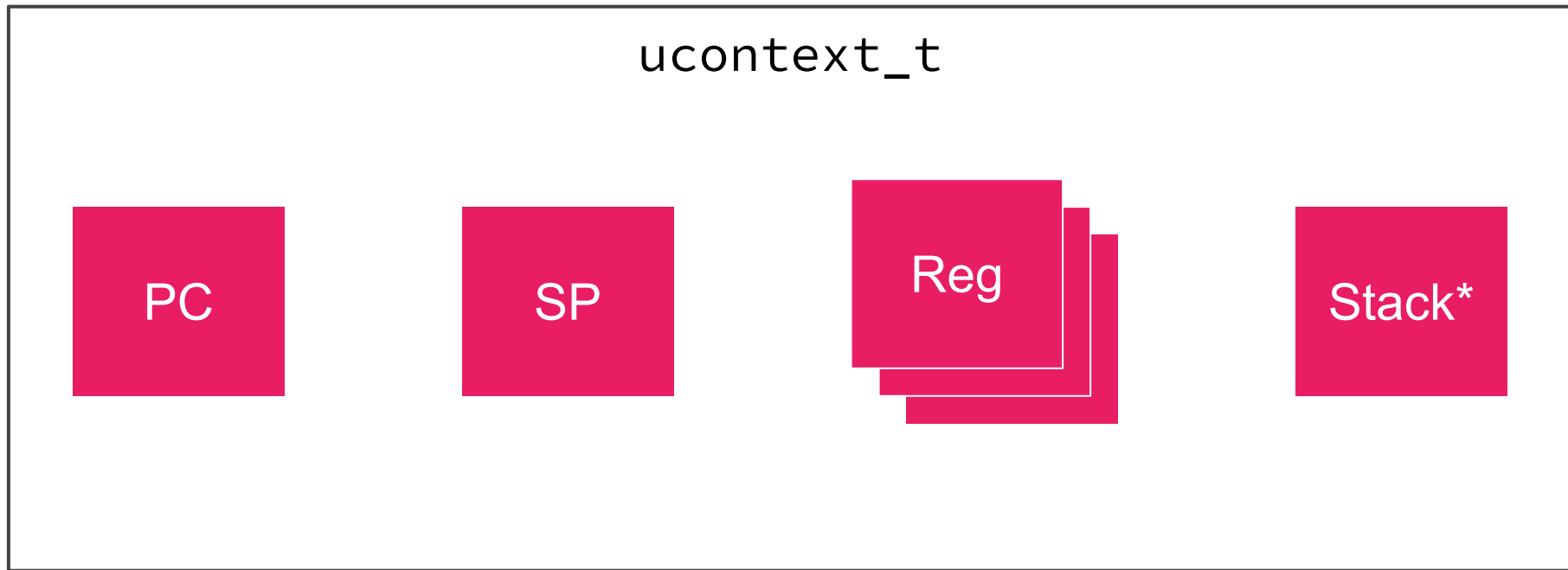
You are going to be implementing

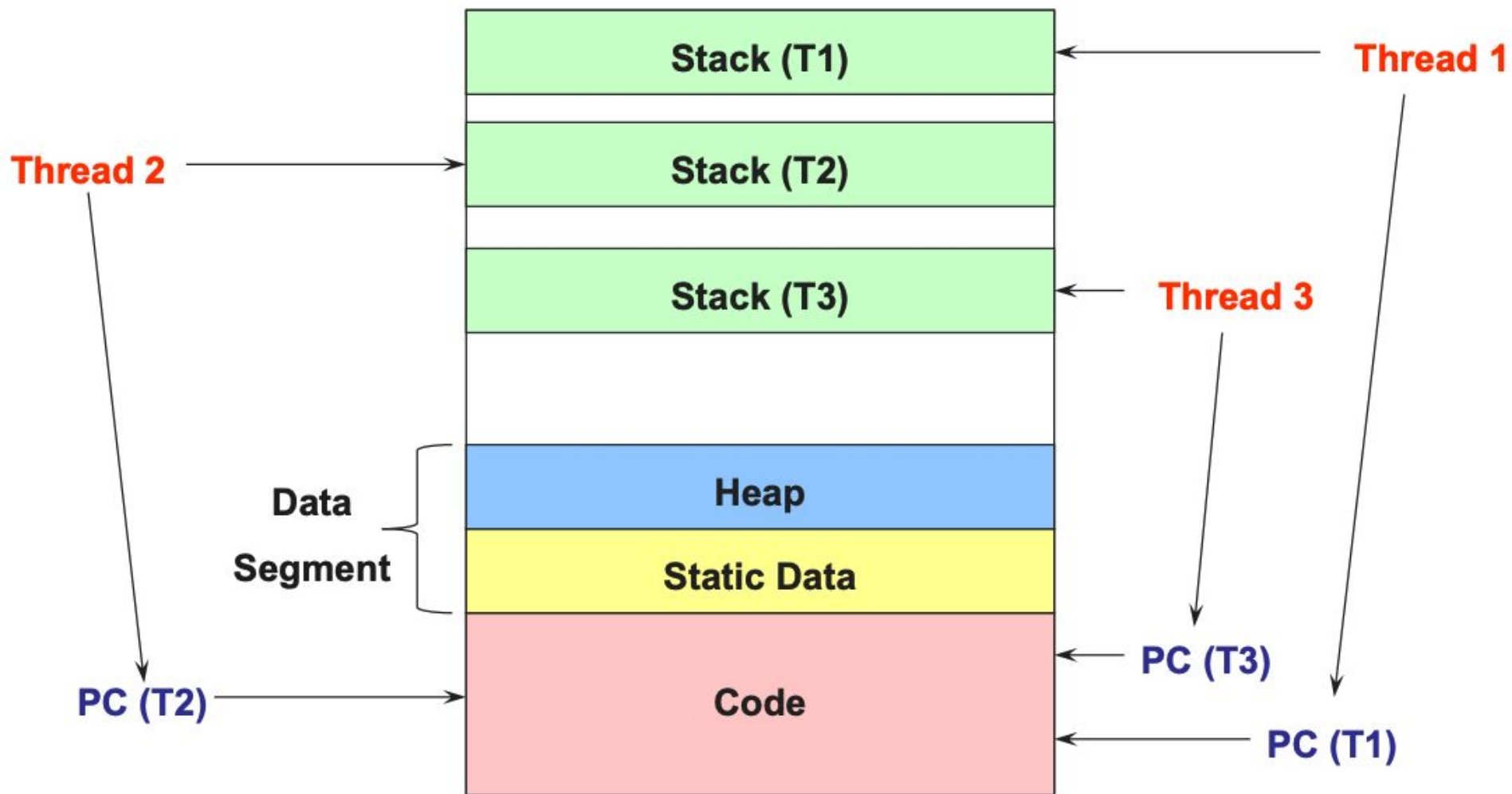
- thread
- mutex
- cv
- join

Remember that a thread's state is a stack, stack pointer, program counter, and a set of registers.

Linux provides `ucontext` as an encapsulation of this state as well as the functionality to `make/swap/get/set` `ucontexts`.

What information do we need to run a thread?





Consider two threads on a single-CPU machine...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```

```
void foo() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```

```
void foo() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```

Consider two threads on a single-CPU machine...

```
int global = 0;
```

```
void bar(int n) {
```

Stack A

printf()

bar() [n]

foo() [i]

```
    printf("n", n);
```

```
    -> 20) {
```

```
void bar(int n) {
```

```
    global ++;
```

```
    printf("%d", global);
```

```
}
```

```
void foo() {
```

```
    for (i = 0; i < 20; i++)
```

```
        bar(i);
```

```
}
```

```
}
```

Stack B

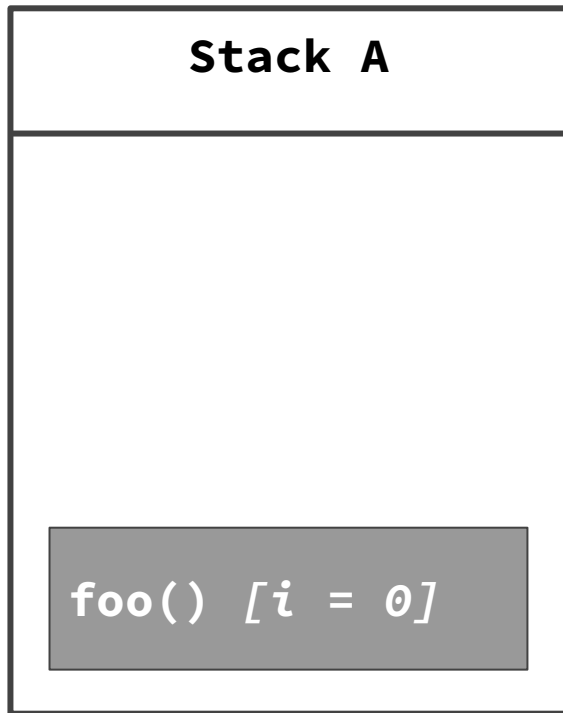
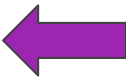
foo() [i]

We run one thread for a while...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```

```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```

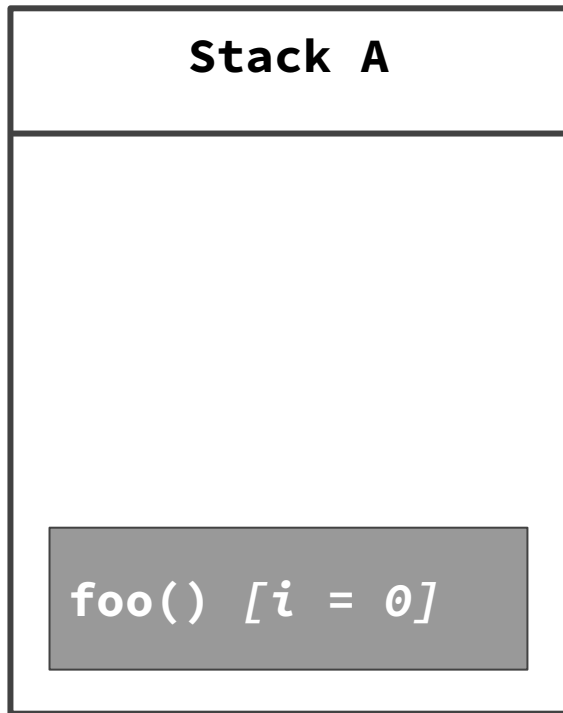
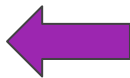


We run one thread for a while...

```
int global = 0;

void bar(int n) {
    global += n
    printf("%i\n", n);
}

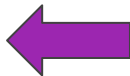
void foo[threadA]() {
    for (i = 0 -> 20) {
        bar(i);
    }
}
```



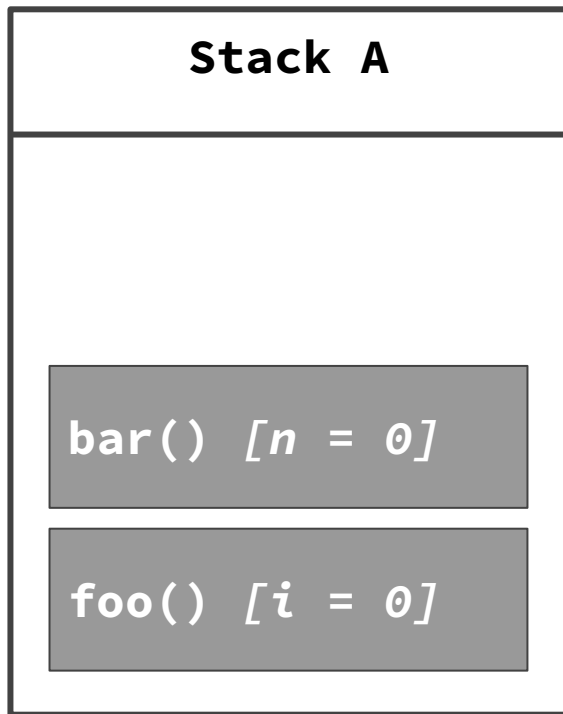
We run one thread for a while...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```



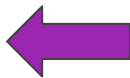
```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```



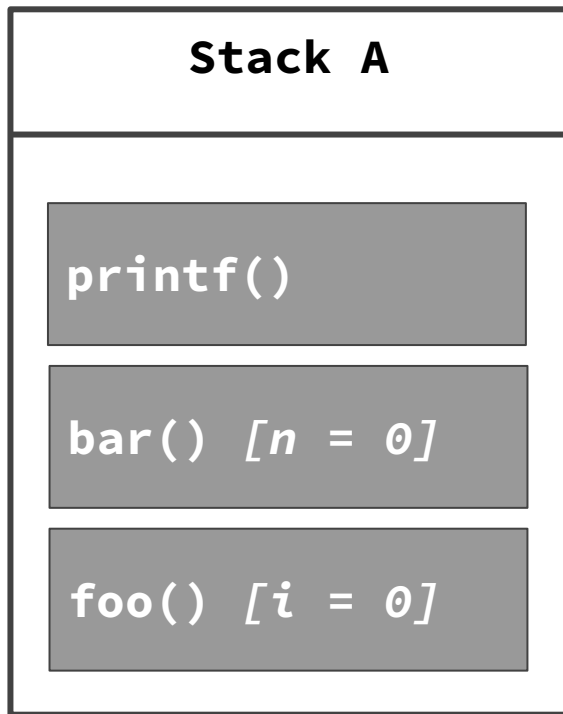
We run one thread for a while...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```



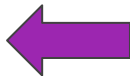
```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```



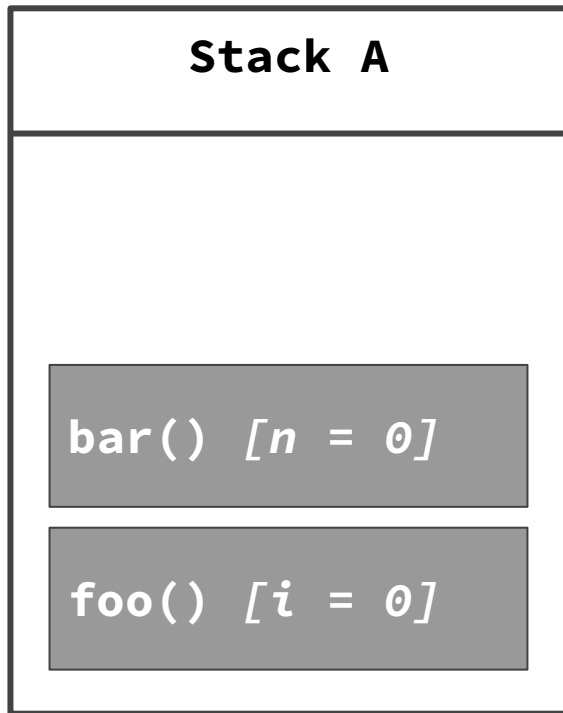
We run one thread for a while...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```



```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```

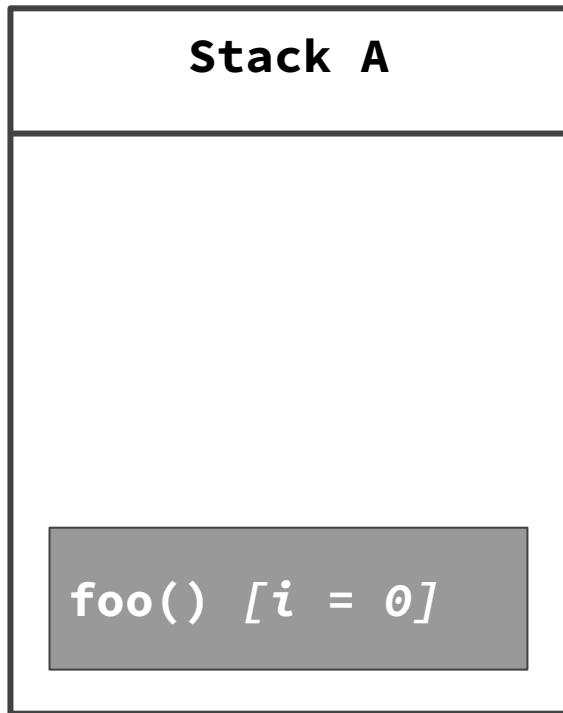
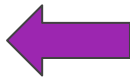


We run one thread for a while...

```
int global = 0;

void bar(int n) {
    global += n
    printf("%i\n", n);
}

void foo[threadA]() {
    for (i = 0 -> 20) {
        bar(i);
    }
}
```

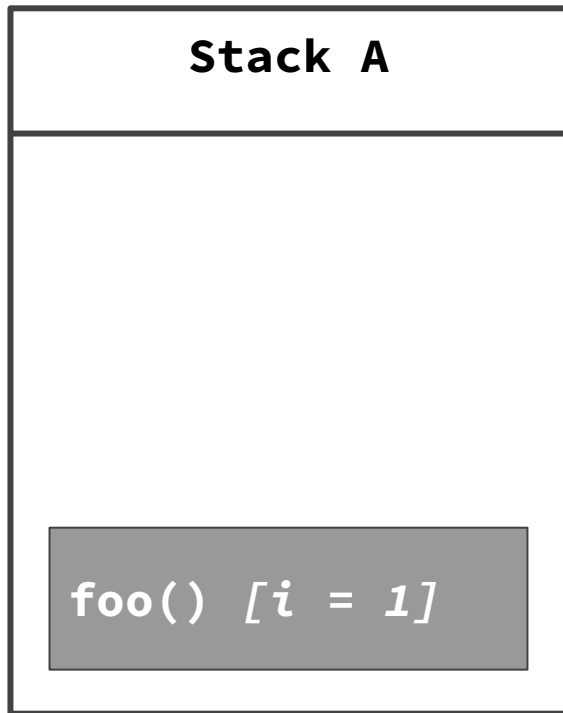
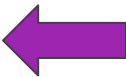


We run one thread for a while...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```

```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```

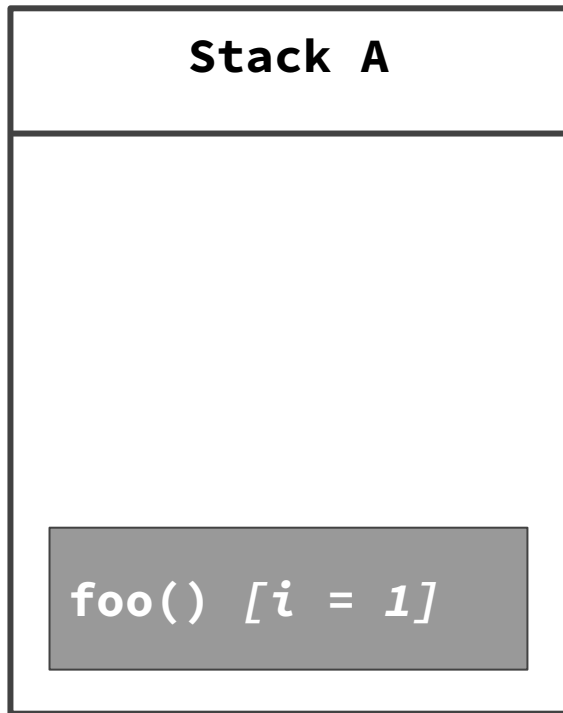
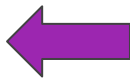


We run one thread for a while...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```

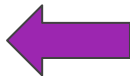
```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```



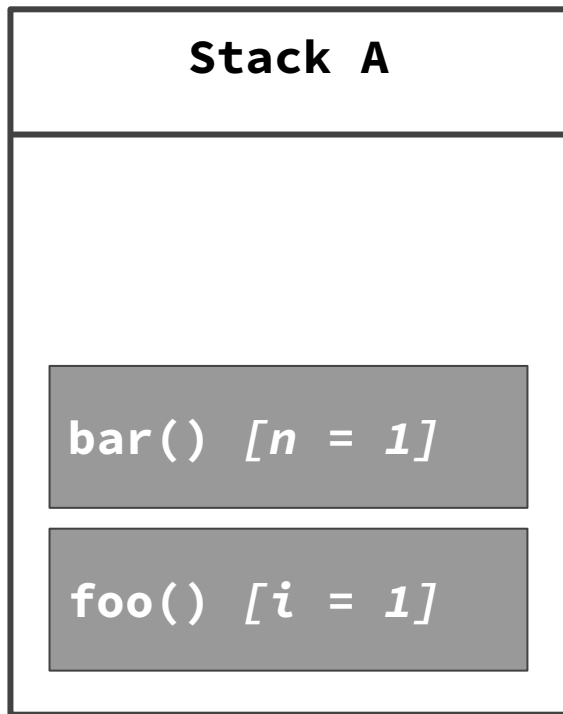
We run one thread for a while...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```



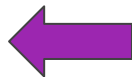
```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```



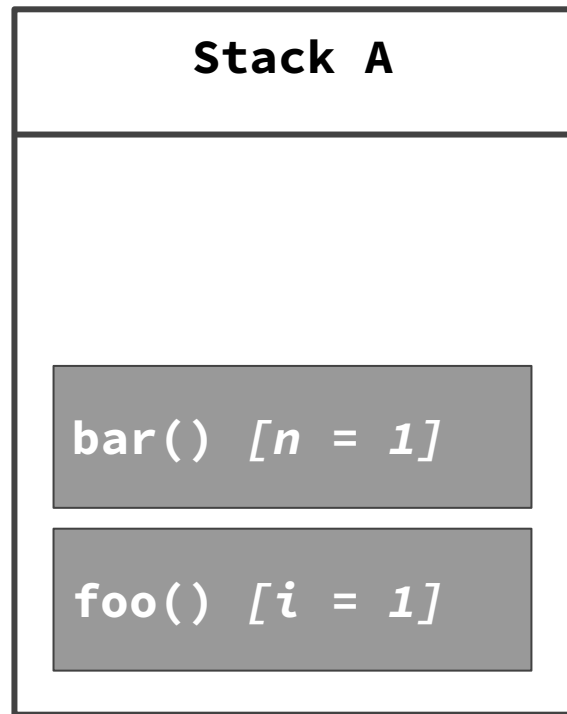
We **Switch!!** while...

```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n);  
}
```



```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```

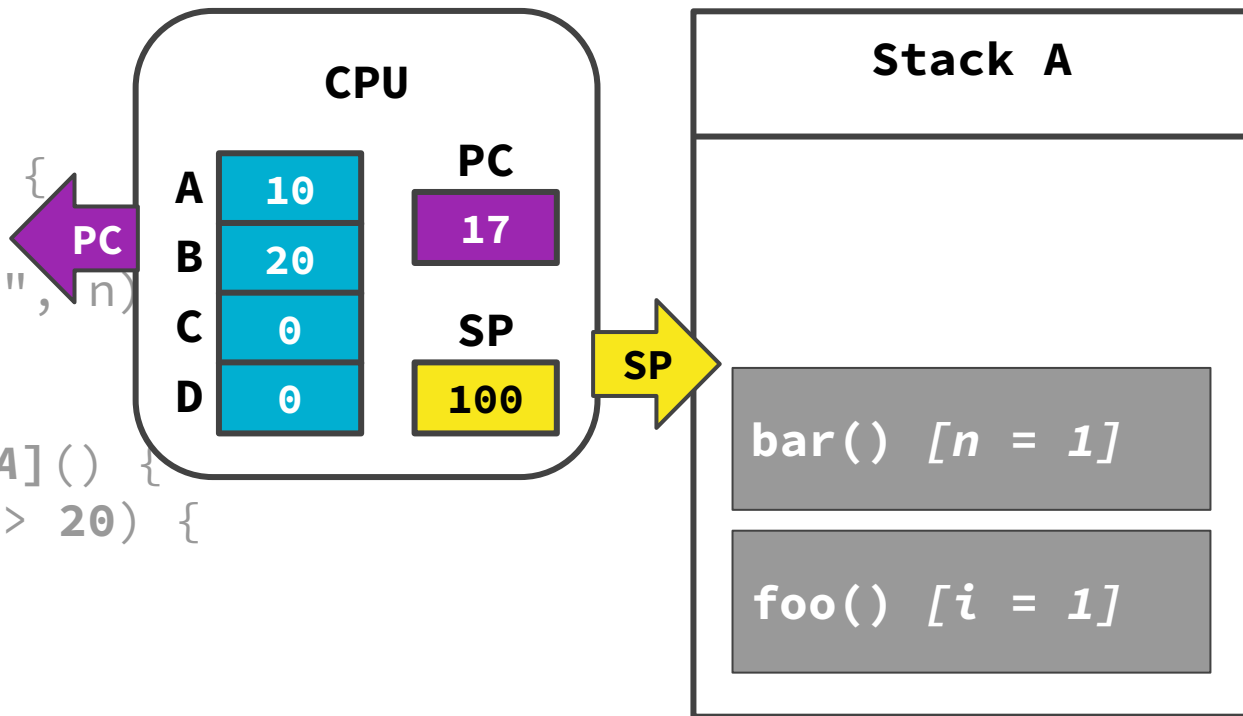


An **execution context** describes what a CPU is doing.

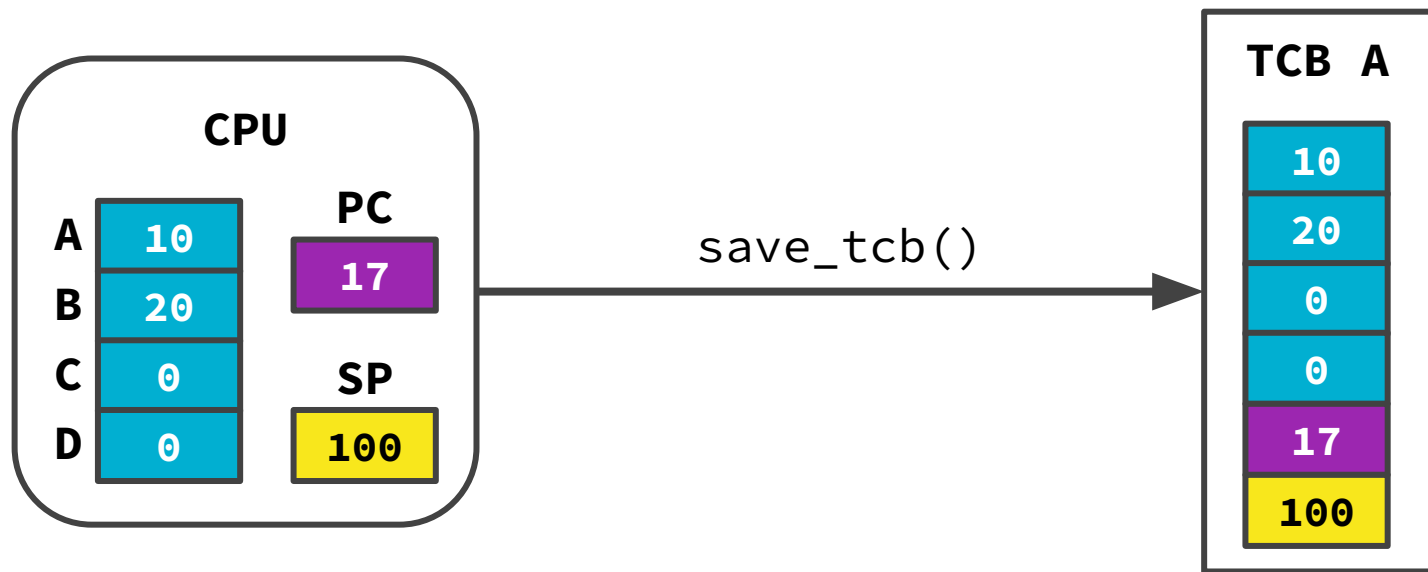
```
int global = 0;
```

```
void bar(int n) {  
    global += n  
    printf("%i\n", n)  
}
```

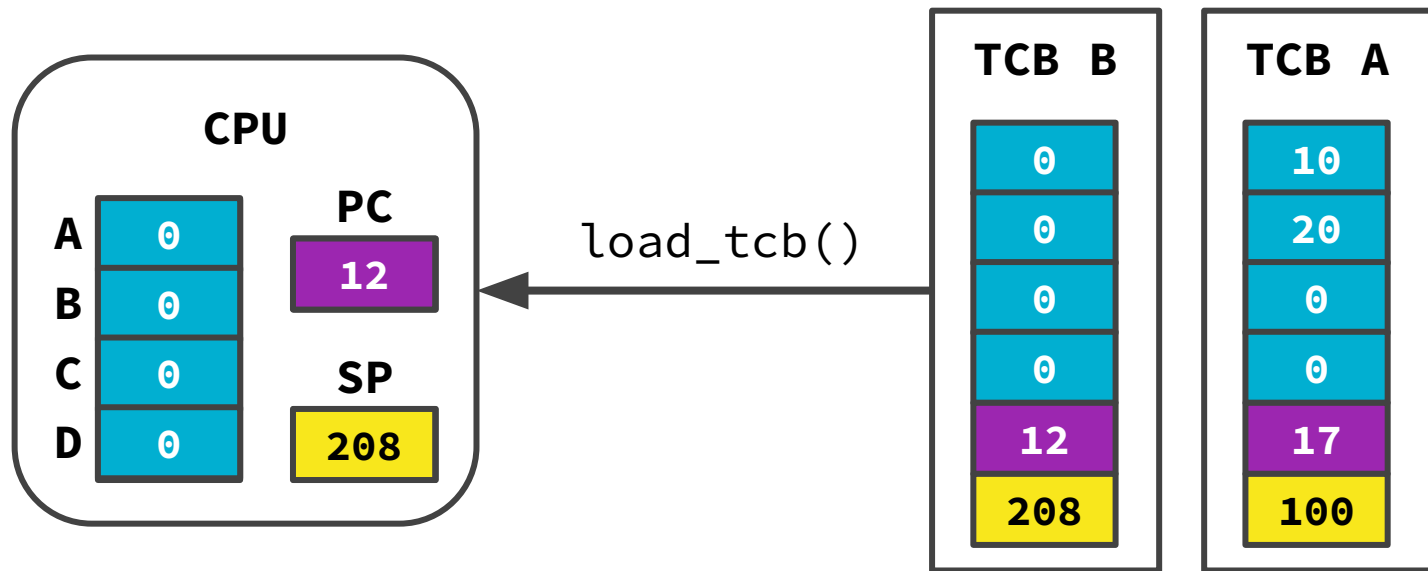
```
void foo[threadA]() {  
    for (i = 0 -> 20) {  
        bar(i);  
    }  
}
```



The TCB stores a **snapshot** of a thread's context.



Contexts can be re-entered by loading a TCB.



TCBs and contexts in Project 2

ucontext_t represents a thread's state

getcontext(*uc_current*)

Saves the currently running context into *uc_current*
(not particularly useful - see hint in spec 3.2)

setcontext(*uc_next*)

Starts to run/resumes running *uc_next*
(doesn't save the currently running context)

TCBs and contexts in Project 2

swapcontext(*uc_current*, *uc_next*)

Effectively *getcontext(current)* and *setcontext(next)*

makecontext(*uc*, *func*, *nargs*, *args...*)

Sets up *uc* such that when you swap to it, it will begin executing code at the beginning of *func*, calling it with *args*

Exercise: is this a correct implementation of swapcontext?

```
void swapcontext(ucontext_t *current, ucontext_t *next) {  
  
    getcontext(current);  
  
    setcontext(next);  
  
}
```

What does this program output?

```
ucontext_t ctxA, ctxB, ctxMain;

void funcA(int arg):
    cout << arg << endl;
    swapcontext(&ctxA, &ctxB);

void funcB(double arg):
    cout << arg << endl;
    swapcontext(&ctxB, &ctxMain);

void main():
    makecontext(&ctxA, funcA, 1, 482);
    makecontext(&ctxB, funcB, 1, 3.14);
    swapcontext(&ctxMain, &ctxA);
    cout << "done" << endl;
```

What does this program output?

```
ucontext_t ctxA, ctxB, ctxMain;

void funcA(int arg):
    cout << arg << endl;
    swapcontext(&ctxA, &ctxB);

void funcB(double arg):
    cout << arg << endl;
    swapcontext(&ctxB, &ctxMain);

void main():
    makecontext(&ctxA, funcA, 1, 482);
    makecontext(&ctxB, funcB, 1, 3.14);
    swapcontext(&ctxMain, &ctxA);
    cout << "done" << endl;
```

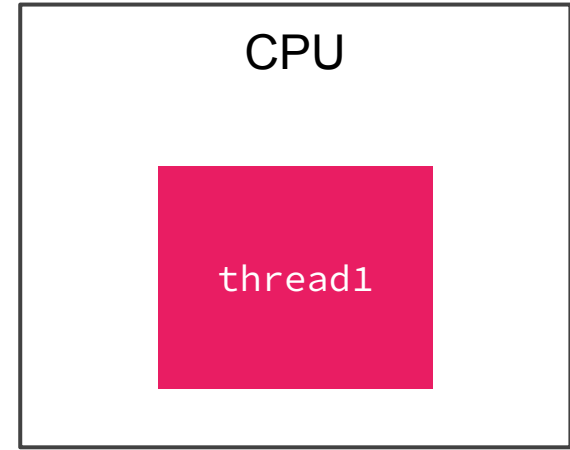
Output:

482

3.14

done

Project 2 Queues Illustration



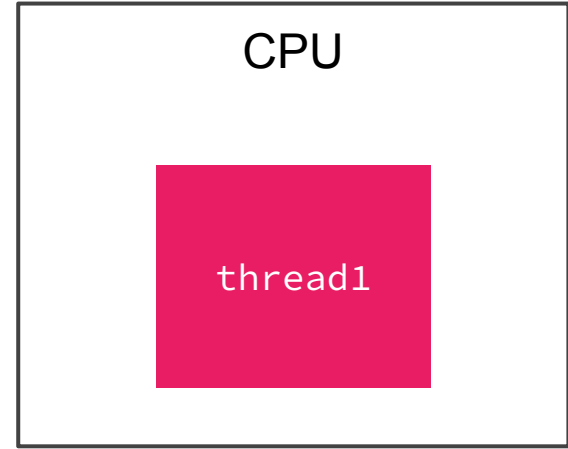
Ready queue



mutexA (unlocked)



thread t2(...)



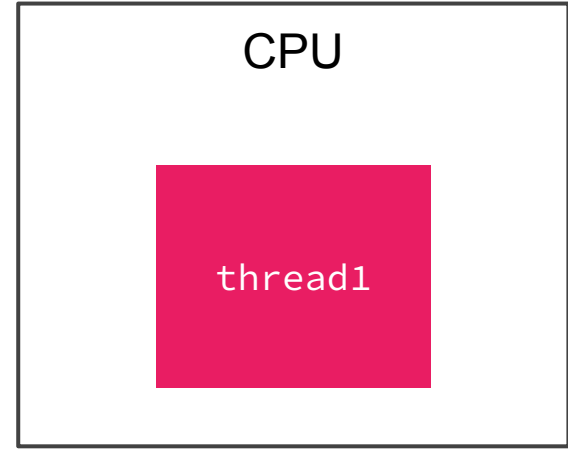
Ready queue



mutexA (unlocked)



thread t2(...)



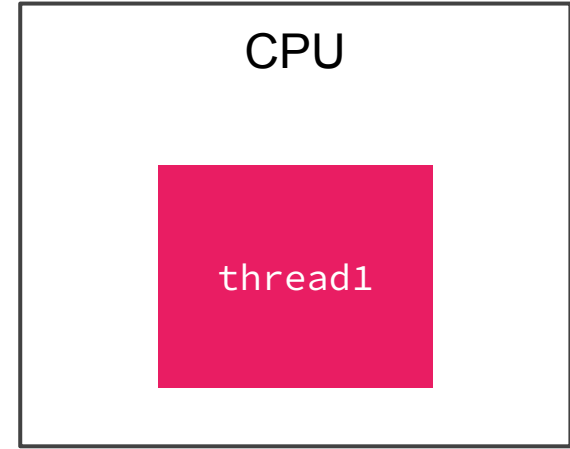
Ready queue



mutexA (unlocked)



`thread::yield()`

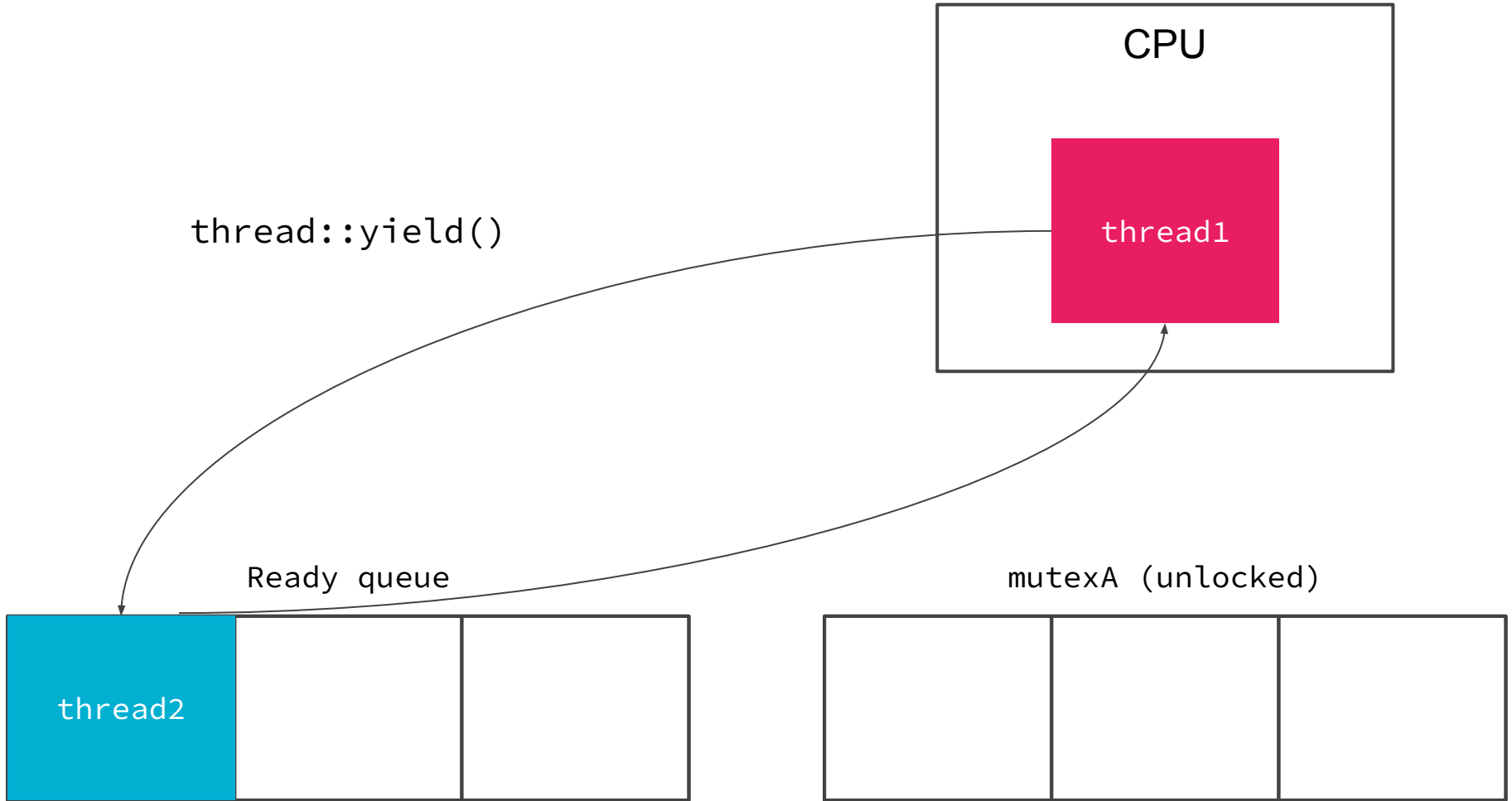


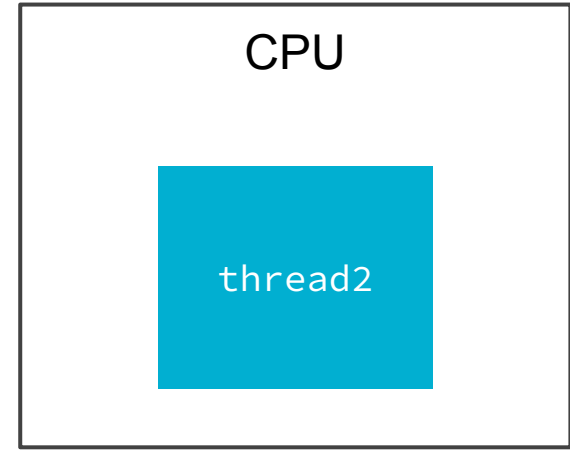
Ready queue



mutexA (unlocked)







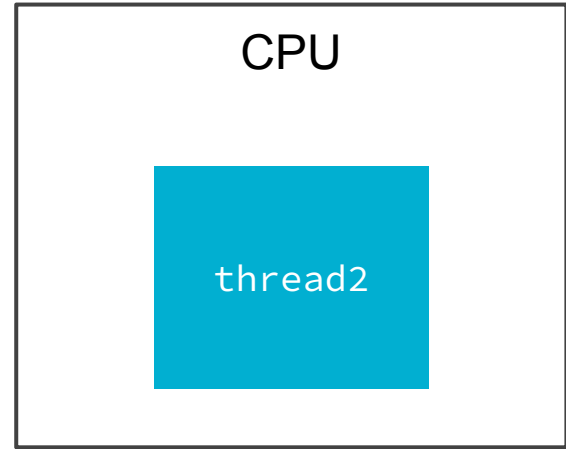
Ready queue



mutexA (unlocked)



`mutexA.lock()`



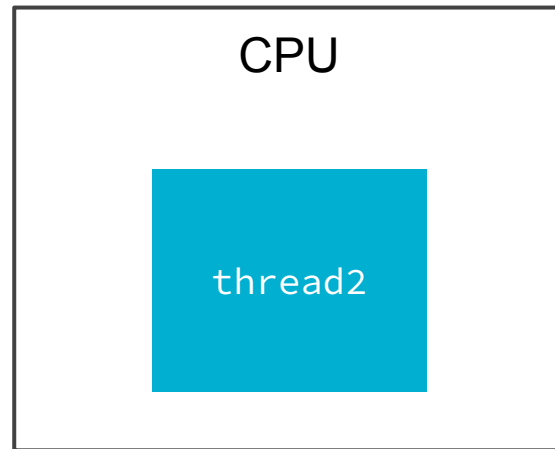
Ready queue



mutexA (unlocked)



`mutexA.lock()`



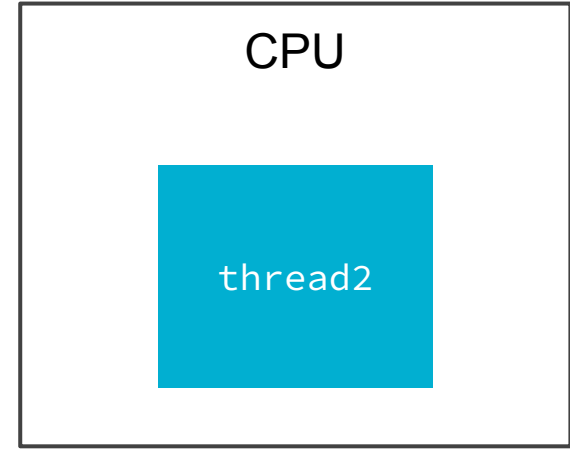
Ready queue



mutexA (locked)



`thread::yield()`

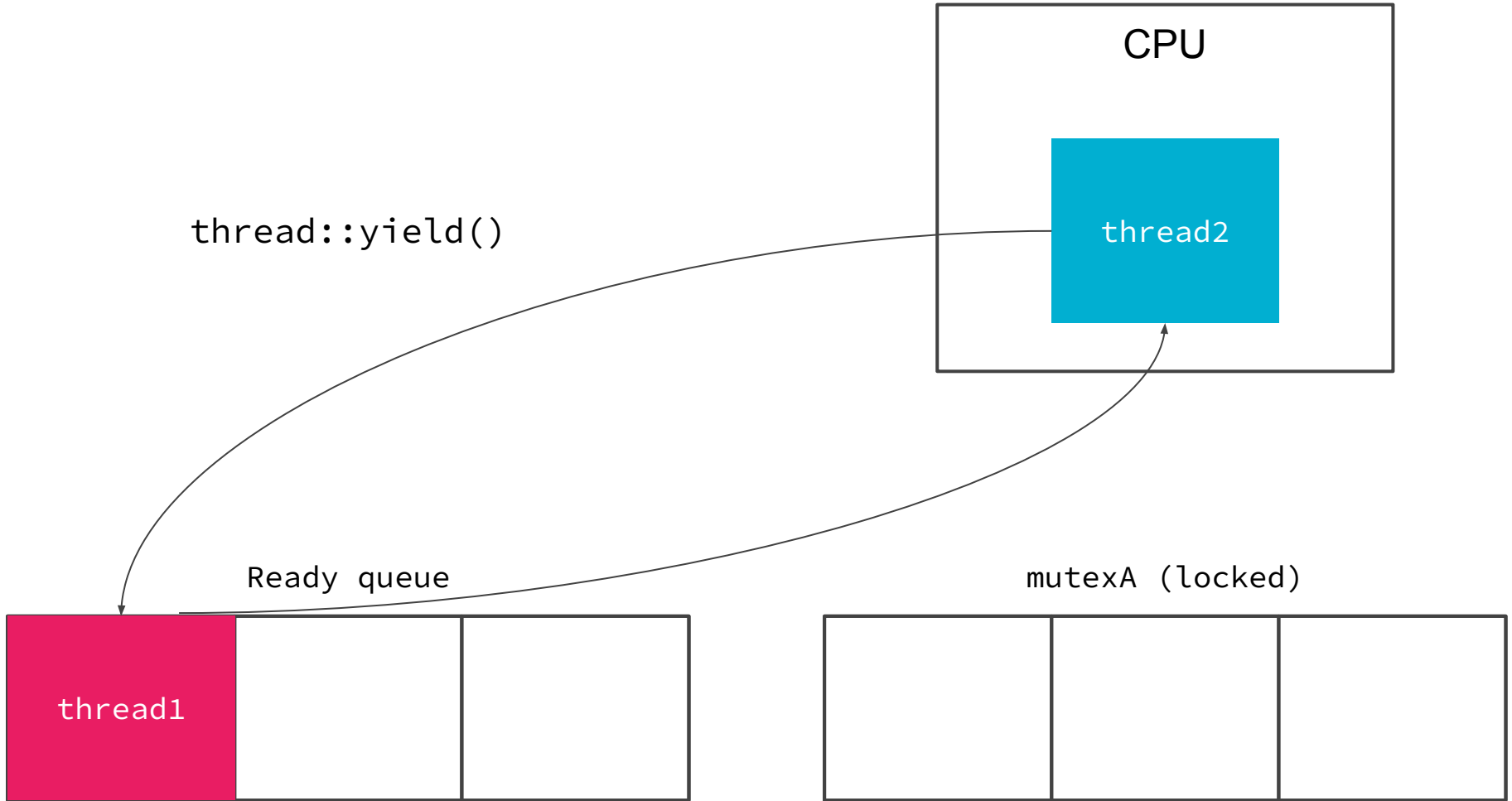


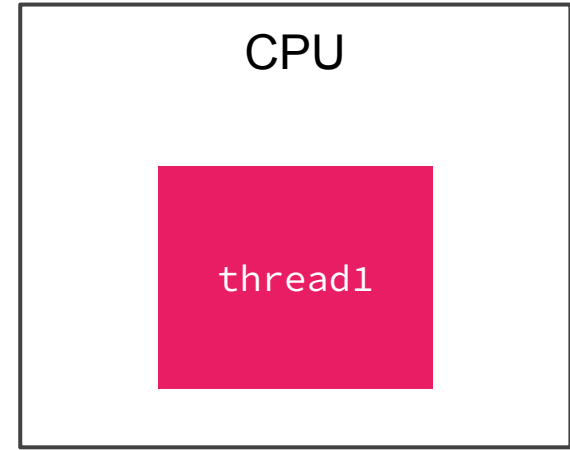
Ready queue



mutexA (locked)







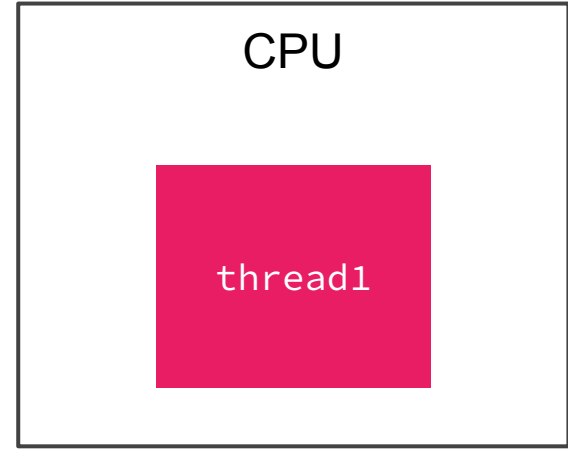
Ready queue



mutexA (locked)



`mutexA.lock()`

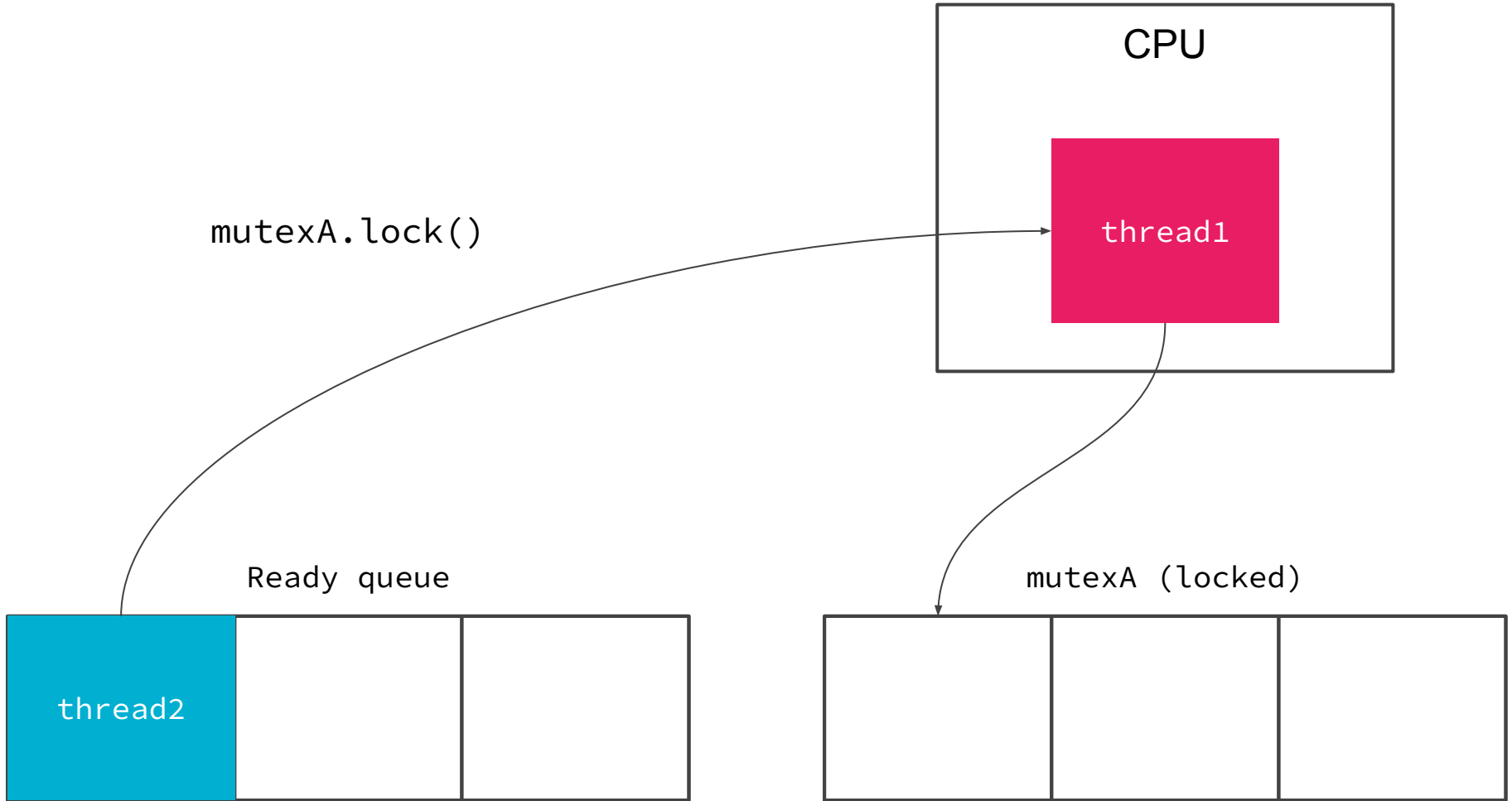


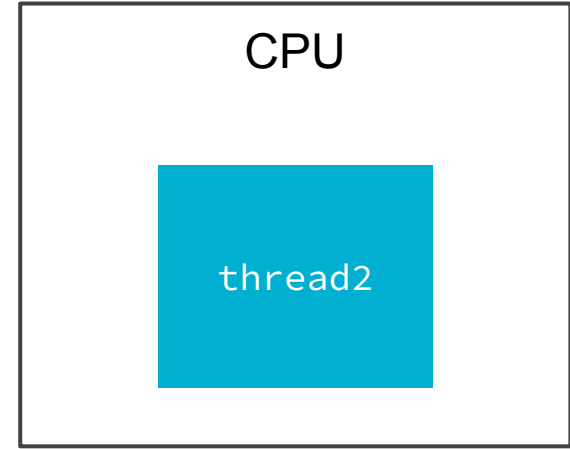
Ready queue



mutexA (locked)







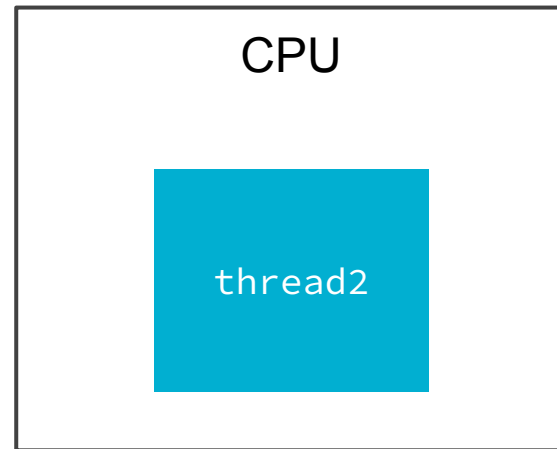
Ready queue



mutexA (locked)



`mutexA.unlock()`



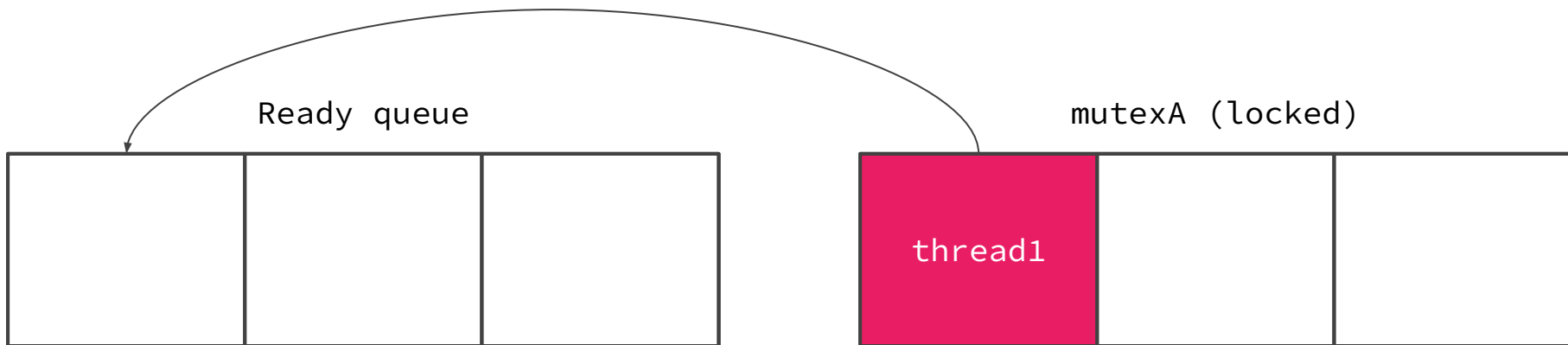
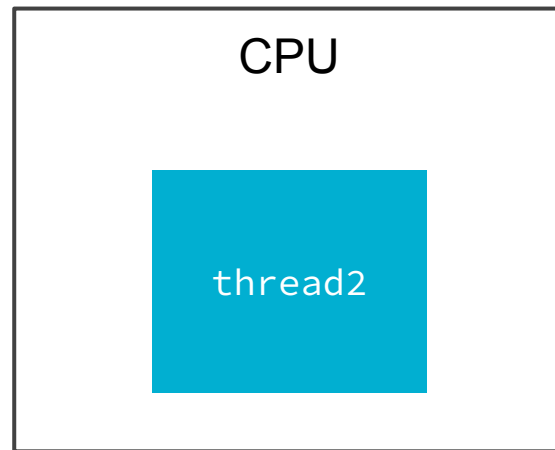
Ready queue

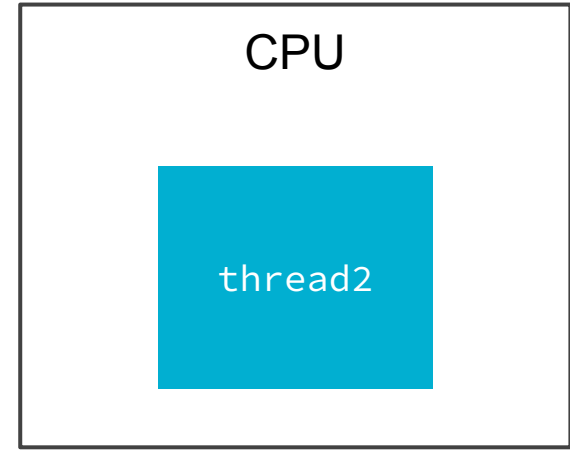


mutexA (locked)



`mutexA.unlock()`





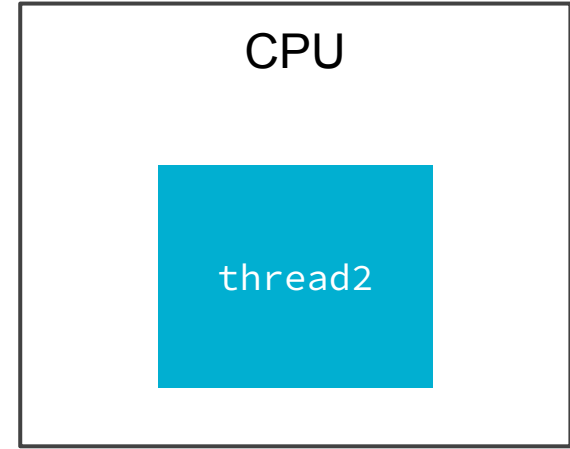
Ready queue



mutexA (locked)



`thread::yield()`

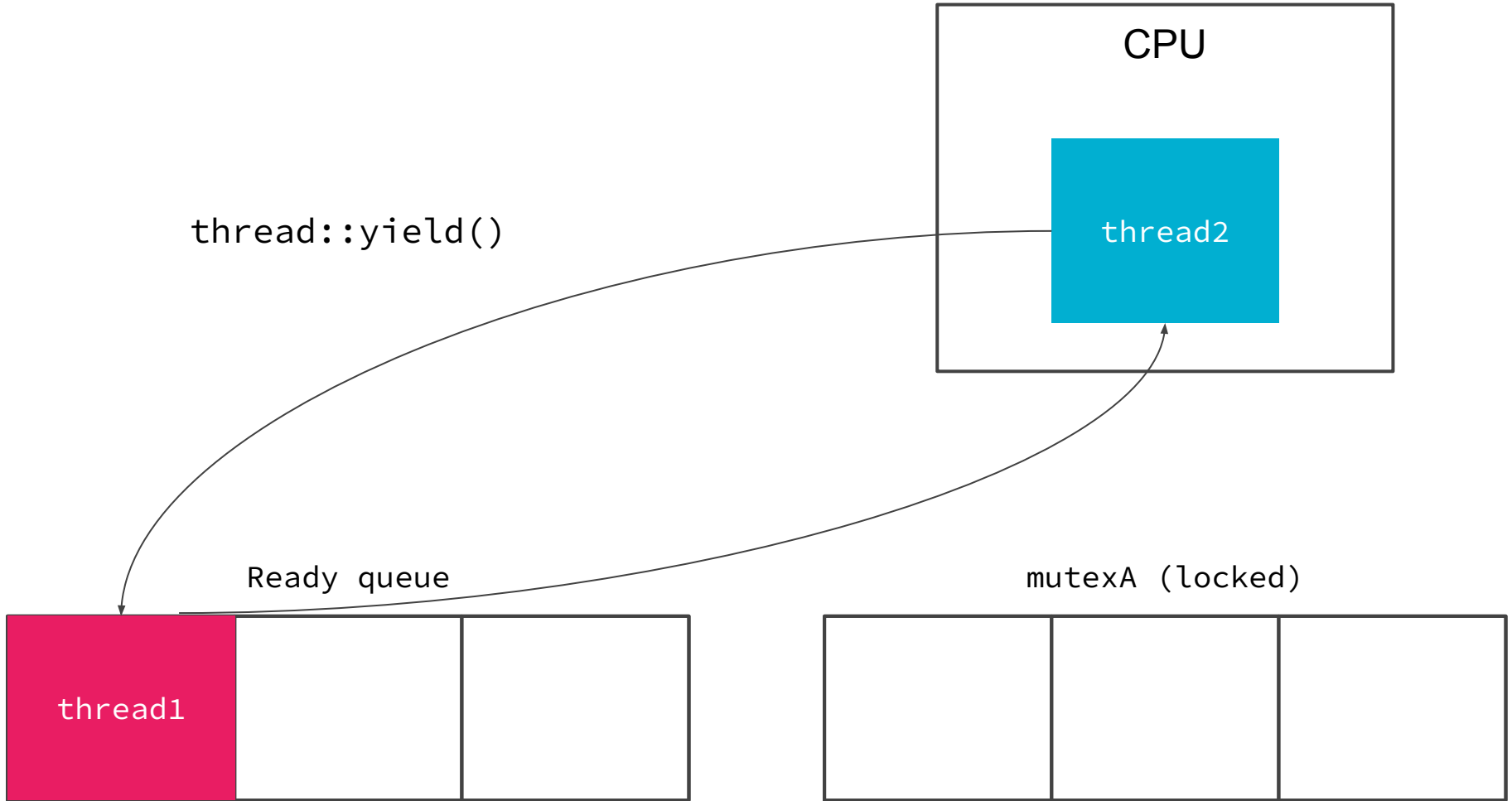


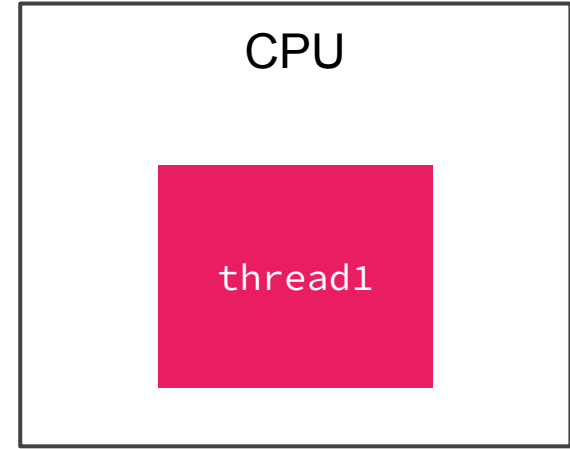
Ready queue



mutexA (locked)







thread1 can now continue
running with the mutex

Ready queue



mutexA (locked)



P2 Exercise

```
mutex mu;
void func_A(uintptr_t) {
    thread t1(func_B, 0);
    thread t2(func_C, 0);
    mu.lock();
    cout << "A has lock" << endl;
    thread::yield();
    mu.unlock();
    cout << "A done" << endl;
}
void func_B(uintptr_t) {
    cout << "B starts" << endl;
    mu.lock();
    cout << "B has lock" << endl;
    mu.unlock();
}
void func_C(uintptr_t) {
    cout << "C starts" << endl;
    thread::yield();
    cout << "C is back" << endl;
    mu.lock();
    cout << "C has lock" << endl;
    mu.unlock();
}
int main(uintptr_t) {
    cpu::boot(1, func_A, 0, false, false, 482);
}
```

With a correct P2 library, what's the output of this program?

P2 Exercise

```
mutex mu;
void func_A(uintptr_t) {
    thread t1(func_B, 0);
    thread t2(func_C, 0);
    mu.lock();
    cout << "A has lock" << endl;
    thread::yield();
    mu.unlock();
    cout << "A done" << endl;
}
void func_B(uintptr_t) {
    cout << "B starts" << endl;
    mu.lock();
    cout << "B has lock" << endl;
    mu.unlock();
}
void func_C(uintptr_t) {
    cout << "C starts" << endl;
    thread::yield();
    cout << "C is back" << endl;
    mu.lock();
    cout << "C has lock" << endl;
    mu.unlock();
}
int main(uintptr_t) {
    cpu::boot(1, func_A, 0, false, false, 482);
}
```

What's the output of this program?

A has lock

B starts

C starts

A done

C is back

B has lock

C has lock

Smart Pointers

Review: Dynamic Memory

- When we want to control the lifetime of an object, we can use dynamic memory
- To use dynamic memory, we use raw pointers
- **operator new** allocates memory for an object
- **operator delete** destroys the object and frees its memory:

```
class Player { ... };  
auto* p = new Player{};  
  
...  
delete p;
```

Dynamic Memory - Ownership

- For dynamically-allocated objects, ownership of an object means it is yours to keep or destroy as you see fit.
- In C++, by ownership, we mean not just which code gets to refer to or use the object, but mostly what code is responsible for **deleting** it
- When using raw pointers, we implement ownership in terms of where in the code we place the delete that destroys the object
 - This is bug prone. What are some possible errors?

Smart Pointers

— — —

- Just as RAII can help us manage the release of resources, is there something that can help us manage the deallocation of dynamic memory?
- **Smart pointers** - class objects that behave like raw pointers but also manage objects that you create with new so that you don't have to worry about when and whether to delete them
- **Acquire memory in constructor; deallocate memory in destructor**
- In C++, `#include <memory>` to use them

```
template<typename T>
class SmartPtr {
public:
    explicit SmartPtr(T *p_in)
        : ptr{ p_in } {}

    ~SmartPtr() { delete ptr; }

    T &operator*() { return *ptr; }

private:
    T *ptr;
};
```

Core Idea: Pointer Abstraction

- A pointer can point to valid or invalid memory
- A pointer that points to valid memory can be changed
- Many issues: Invalid Accesses, Memory Leaks

Smart Pointers are **guaranteed to point to valid memory**, and **will clean up resources** when destroyed.

Why smart pointers?

- **Ease of use:** They handle deallocation for you, helping you avoid memory leaks and use-after-free bugs
- **Familiarity with the STL:** They are widely used in C++ codebases and it is be useful to have experience with them
- **Avoiding Bugs:** They can help you avoid bugs by enforcing certain invariants (e.g. unique ownership)

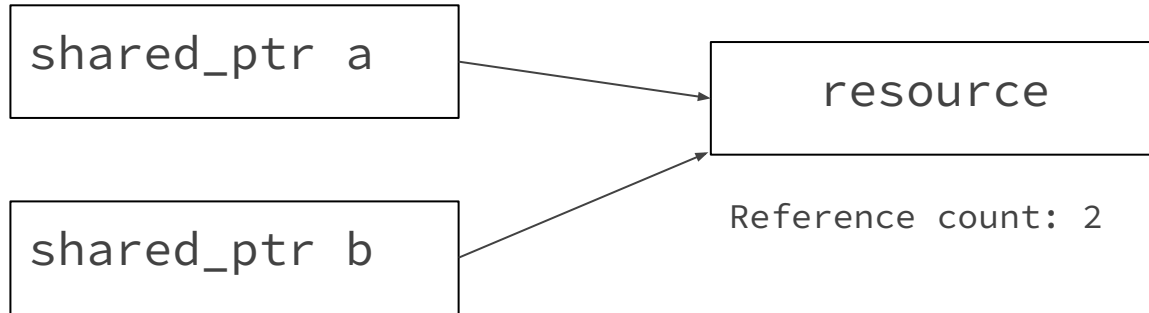
Smart Pointers

— — —

- **shared_ptr<T>**
 - Implements **shared ownership**.
 - Any number of shared_ptrs can jointly own an object. When the last shared_ptr pointing to an object is destroyed, the object is **automatically destroyed**.
- **unique_ptr<T>**
 - Implements **unique ownership**.
 - Manages memory like a shared_ptr, but there can only be **one** unique_ptr to a given object at a time.
 - Can't be copied – requires **move semantics**.

shared_ptr<T>

- Multiple shared pointers can point to a resource
- When all shared pointers are destroyed, resource is released
- Maintains a “reference count” for the resource



shared_ptr<T> Example

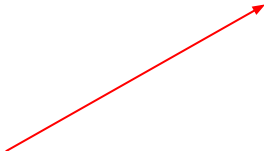
```
{
shared_ptr<int> b;
{
    shared_ptr<int> a = make_shared(10); //resource allocated
    b = a; //copy of shared pointer
}
//a is destroyed
}
//b is destroyed
//resource freed
```

unique_ptr<T>

- Ensures only one owner for a given resource
- **Cannot be copied**, ownership must be transferred from one scope to another using `std::move()`



No other unique ptr



unique_ptr<T> Example

```
{  
unique_ptr<int> b;  
    {  
        unique_ptr<int> a = make_unique(10); //resource allocated  
        b = std::move(a); //must move instead of copy  
    }  
}  
//b is destroyed  
//resource freed
```

Best Practices

— — —

- You can use `.get()` to obtain the raw pointer value of a smart pointer, but BE CAREFUL— this raw pointer does *not* provide the guarantees that smart pointers do (such as the assurance of no dangling pointers)
- Unless you need to ensure single ownership, start with `shared_ptr`

Read:

https://en.cppreference.com/w/cpp/memory/unique_ptr

https://en.cppreference.com/w/cpp/memory/shared_ptr

Converting Code: Raw Pointers

```
struct big { //expensive to copy
```

```
    ...
```

```
}
```

```
big* a = new big();
```

```
big* b = a;
```

```
delete b; //needed to not leak memory
```

```
//a and b still accessible!
```

```
*a // segfault or worse
```

Converting Code: Smart Pointers

```
struct big {    //expensive to copy
    ...
}
shared_ptr<big> a = make_shared<big>();
{
    shared_ptr<big> b = a; //copy
}
//impossible to have invalid access
//resource still cleaned up
```

Project 2 Tips

Avoiding Cyclic Dependencies

— — —

1. Include the definition (implementation) in the .cpp file and never include the .cpp files.
2. Keep the function/variable declaration (not the implementation) in the .h file.
3. Split up files that define new structs from files that have functions that use that struct.

Forward Declaring

— — —

```
// File1.h:  
#include "File2.h"  
// definition  
class Foo {  
    int x;  
};  
void func(bar my_bar);
```

```
// File 2.h  
class Foo;   
  
class Bar {  
    int y;  
};  
  
void func(foo* my_foo);  
void func2(const foo& my_foo2);  
  
// File2.cpp  
#include "File1.h"  
void func(foo* my_foo) {  
    ...  
}  
void func2(const foo& my_foo2) {  
    ...  
}
```

Static Member Variables

```
// In the .h  
class Foo {  
    static int bar;  
};
```

```
// In the .cpp  
int Foo::bar;
```

Thread lifetimes

Thread execution != thread object.

If thread object goes out of scope / gets deleted, thread execution still continues.

Is it safe to store a **thread*** in your ready queue / etc?

- Who controls the lifetime of the thread object? User code or thread library code?

Recall from Lab 01

```
void func(uintptr_t a) { <-- func still runs (or maybe it finished already)  
    cout << a << endl;  
}  
// Called from cpu::boot()  
void main_thread(uintptr_t) {  
    // Spawn 4 threads and give each one a different num  
    for (int i = 0; i < 4; ++i) {  
        uintptr_t num = i * 42;  
        thread t(func, num);  
    }  
} <-- t destroyed here
```

CPUs in the Project 2 infrastructure

```
cpu::boot(4, func, arg, ...)
```

```
graph TD; A["cpu::boot(4, func, arg, ...)"] --> B["pthread<br/>cpu(0, 0);"]; A --> C["pthread<br/>cpu(func, arg);"]; A --> D["pthread<br/>cpu(0, 0);"]; A --> E["pthread<br/>cpu(0, 0);"];
```

The diagram illustrates the execution of the `cpu::boot(4, func, arg, ...)` function. This function spawns four parallel threads, each represented by a rounded rectangle. Each thread is a `pthread` and contains a `cpu` function call. The first thread calls `cpu(0, 0);`, the second thread calls `cpu(func, arg);`, and the third and fourth threads call `cpu(0, 0);`. Arrows point from the `cpu::boot` function box to each of the four thread boxes.

pthread

`cpu(0, 0);`

pthread

`cpu(func, arg);`

pthread

`cpu(0, 0);`

pthread

`cpu(0, 0);`

P2 tip: switching cpus in gdb

P2 infrastructure simulates cpus using real pthreads.

You can see and switch simulated cpus using:

```
(gdb) $ info threads / (lldb) $ threads list
```

```
(gdb) $ thread <N> / (lldb) $ thread select <N>
```

P2 tip: printing output

****don't use cout in both the thread library and test cases****

You could deadlock on cout's internal mutex.

Thread library: use **printf**

User code: use **cout**

Project 2 Testing

Testing Tips

- Test cases you *submit* would have interrupts disabled
 - How to test different interleavings of threads?
 - Use `thread::yield!`
- Create test cases that test "0, 1, and many"
 - For example, write a test case that tests 0 threads on the join queue, 1 thread on the join queue, and many threads on the join queue
 - Testing just 1 thread on the join queue doesn't test the ordering that those threads will be run after the joined thread finishes
- Keep tests as small and pointed as possible!
- Check every function's basic functionalities
- Have a documentation of what functionalities each test case is testing for

Interrupts

- How can we test a thread library to make sure it works in the presence of interrupts?
- It's most helpful to test the edge cases
 - What happens if an interrupt arrives **immediately before** calling `interrupt_disable()`?
 - What happens if an interrupt arrives **immediately after** calling `interrupt_enable()`?
 - You can (and should) test both by explicitly calling your timer interrupt handler in these places
- Remember that interrupts should always be enabled in user code.
 - Check this with `assert_interrupts_enabled()`

Error Handling

- Carefully read Section 3.8 of the project spec and test all the cases mentioned there
- When writing test cases that throw an exception, make sure to catch it
 - Test cases submitted to AG to expose instructor buggy solutions cannot exit with error
 - Use a try/catch and print something out to indicate that an exception was thrown

Catch the exception in test case

— — —

```
intptr_t i = 0;
while(true){
    try {
        thread(func, reinterpret_cast<void *>(i++));
    }
    catch(std::bad_alloc &ba){
        ...
    }
}
```

Error Handling

— — —

- Misuse of thread functions
 - Calling `unlock()` from a thread that does not own a mutex should throw a `std::runtime_error`
- Resource exhaustion
 - Operator `new` will throw a `std::bad_alloc` if it cannot allocate memory
 - Remember that these exceptions propagate to user code.

Questions?